

Managing the Conversation History

Managing the Conversation History

- One important concept to understand when building chatbots is how to manage conversation history. If left unmanaged, the list of messages will grow unbounded and potentially overflow the context window of the LLM.
- Therefore, it is important to add a step that **limits the size of the messages** you are passing in.
- `'trim_messages'` helper to reduce how many messages we're sending to the model.
- The trimmer allows us to specify how many tokens we want to keep, along with other parameters like if we want to always keep the system message and whether to allow partial messages

```
from langchain_core.messages import SystemMessage, trim_messages, AIMessage

trimmer=trim_messages(
    max_tokens=45,
    strategy="last", # Default = "last"
    token_counter=model,
    include_system=True, # Default = False
    allow_partial=False, # Default = False
    start_on="human"
```

- `SystemMessage` : For system-level instructions to the AI
- `trim_messages` : Function to truncate message history

Analogy:

Think of `trim_messages()` like a **paper shredder with rules**:

- It decides **where to start cutting**
- How many tokens to keep
- Whether to include system messages
- Whether it can cut **inside** a message (partial cutting)

Parameter	Meaning
<code>max_tokens=45</code>	The total allowed tokens in the output. Trim down to 45 tokens or less.
<code>strategy="last"</code>	Keep the last messages and drop the older ones.
<code>token_counter=model</code>	Uses the tokenizer of the LLM (e.g., <code>ChatGroq</code>) to count tokens.
<code>include_system=True</code>	Keep the initial <code>SystemMessage</code> too.
<code>allow_partial=False</code>	Don't cut a message in the middle . Either keep it or skip it.
<code>start_on="human"</code>	Start trimming from the first Human message.

```

messages = [
    SystemMessage(content="you're a good assistant"), # [SYSTEM]
    HumanMessage(content="hi! I'm bob"),             # [HUMAN]
    AIMessage(content="hi!"),                         # [AI]
    HumanMessage(content="I like vanilla ice cream"), # [HUMAN]
    AIMessage(content="nice"),                       # [AI]
    HumanMessage(content="whats 2 + 2"),              # [HUMAN]
    AIMessage(content="4"),                          # [AI]
    HumanMessage(content="thanks"),                   # [HUMAN]
    AIMessage(content="no problem!"),                 # [AI]
    HumanMessage(content="having fun?"),              # [HUMAN]
    AIMessage(content="yes!"),                       # [AI]
]

```

```
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 2.89kB/s]

d:\Python_Env\LangChain\venv\lib\site-packages\huggingface_hub\file_download.py:143: UserWarning
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as a
warnings.warn(message)

vocab.json: 100% ██████████ 1.04M/1.04M [00:00<00:00, 1.92MB/s]
merges.txt: 100% ██████████ 456k/456k [00:00<00:00, 11.2MB/s]
tokenizer.json: 100% ██████████ 1.36M/1.36M [00:00<00:00, 6.12MB/s]
config.json: 100% ██████████ 665/665 [00:00<00:00, 80.8kB/s]

[SystemMessage(content="you're a good assistant", additional_kwargs={}, response_metadata={}),
 HumanMessage(content='I like vanilla ice cream', additional_kwargs={}, response_metadata={}),
 AIMessage(content='nice', additional_kwargs={}, response_metadata={}),
 HumanMessage(content='whats 2 + 2', additional_kwargs={}, response_metadata={}),
 AIMessage(content='4', additional_kwargs={}, response_metadata={}),
 HumanMessage(content='thanks', additional_kwargs={}, response_metadata={}),
 AIMessage(content='no problem!', additional_kwargs={}, response_metadata={}),
 HumanMessage(content='having fun?', additional_kwargs={}, response_metadata={}),
 AIMessage(content='yes!', additional_kwargs={}, response_metadata={})]
```



It skipped first 2 messages because `max_tokens=45`.

If all messages together exceed 45 tokens:

- The trimmer will drop the **earliest** messages (starting from first Human, because of `start_on="human"`)
- It will try to keep **as many whole messages as possible**, until reaching 45 tokens
- It **includes** the `SystemMessage` , because `include_system=True`

Pass trimmer in a chain

```
prompt= ChatPromptTemplate.from_messages(
    [
```

```
(
    "system", "Answer all questions to the best of your ability in {language}.",
    MessagesPlaceholder("messages")

)
```

```
from operator import itemgetter

from langchain_core.runnables import RunnablePassthrough

chain=(
    RunnablePassthrough.assign(messages=itemgetter("messages")|trimmer)
    | prompt
    | model
)

response=chain.invoke(
    {
        "messages":messages + [HumanMessage(content="What ice cream do i like")],
        "language":"English"
    }
)
response.content
```

```
"As an AI, I don't have access to your personal information, including your ice cream preferences."
```



It trimmed the “**vanilla ice-cream**” message because **max_tokens=45** .

- **RunnablePassthrough** : A special object that just **passes input as-is** unless you assign transformations to it.
 - **.assign(...)** : After transforming the **messages** , it **replaces** the **"messages"** field in the input dictionary with the trimmed version.
 - **itemgetter("messages")** : Extracts the messages list from input. (Gets the **"messages"** key from a dictionary.)
 - **| trimmer** : Pipes those messages through the trimmer to reduce token count.

messages + [...] :

- Adds a new question to the conversation.

If we set 🖱️ **max_tokens=70**

```
response=chain.invoke(  
    {  
        "messages":messages + [HumanMessage(content="What ice cream do i like")],  
        "language":"English"  
    }  
)  
response.content
```

```
'You said you like vanilla ice cream! 😊 \n'
```

- It answers the question because the token limit has not trimmed the message.

```

response = chain.invoke(
    {
        "messages": messages + [HumanMessage(content="what math problem
did i ask")],
        "language": "English",
    }
)
response.content

```

```
'You asked "what is 2 + 2".\n'
```

Wrap this in the Message History

```

with_message_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="messages"
)
config={"configurable":{"session_id":"chat5"}}

```

```

response = with_message_history.invoke(
    {
        "messages": messages + [HumanMessage(content="whats my nam
e?")],
        "language": "English",
    },
    config=config,
)

response.content

```

```
'Your name is Bob! 😊 \n\n\n\nI remember things we talk about.\n'
```

```
messages = [  
    SystemMessage(content="you're a good assistant"),  
    HumanMessage(content="hi! I'm bob"),  
    AIMessage(content="hi!"),  
    HumanMessage(content="I like vanilla ice cream"),  
    AIMessage(content="nice"),  
    HumanMessage(content="whats 2 + 2"),  
    AIMessage(content="4"),  
    HumanMessage(content="thanks"),  
    AIMessage(content="no problem!"),  
    HumanMessage(content="having fun?"),  
    AIMessage(content="yes!")  
]
```